District heating pipe network layout using minimum spanning trees

The complete explanation: what the project is, how every part works, what we found, and how to present it

CLL798D / CHL7903 Network Science · Final presentation, 12 November 2026

Uday Pratap Singh · Hardik Kotwar

Contents

1. The project in two minutes
2. Background: district heating as a graph problem
3. The data and how the network is built
4. Edge weights: map distance and terrain
5. The tree algorithms
6. The cost model (the physics)
7. Pressure zones
8. Reliability: bridges, N-1 and expected unserved demand
9. What we showed at the prelim
10. After the prelim: the five new results (slides 3 to 7)
11. Summary slide, limitations and the main message
12. Question bank for the Q&A
13. Glossary
14. One-page cheat sheet

1. The project in two minutes

A district heating network carries hot water from one central plant to many buildings through buried pipes. Before building it, someone has to decide **which pipes to lay**: which substation connects to which. That decision is a graph problem. The plant and the substations are **nodes**, every possible pipe is an **edge**, and the network we build is a **subgraph** that connects everything.

The classic answer, used by our reference paper (Gao et al., 2020), is the **minimum spanning tree (MST)**: the set of edges that connects all nodes with the smallest total length and no loops. Our project asks whether that answer is good enough for a real network, using two real German districts: **Bremen** (flat) and **Stuttgart** (hilly).

At the prelim (9 September) we showed three things on one 40-substation network per city:

- Weighting edges by terrain (slope makes digging more expensive) gave a tree that was **3% cheaper** in hilly Stuttgart and changed nothing in flat Bremen.
- In Stuttgart the height difference is too large for one set of pipes to hold the water pressure, so the network needs **2 pressure zones**.
- A tree has no backup: if one pipe breaks, everything after it loses heat. **One extra pipe**, chosen greedily, removed 57% of the expected unserved demand.

After the prelim we tested these claims much harder and added a new method:

#	What we did	What we found
1	Rebuilt the network 120 different ways (6 sizes × 10 clusterings × 2 cities)	The 3% saving is real on average but very noisy: −6% to +13% for the same size
2	An edge-swap search that improves the MST using the real cost	6 to 9% cheaper than the MST in both cities, even flat Bremen
3	Checked which graph property predicts cost	Cost follows the distance from the plant (r = 0.98), not total length
4	Scaled Stuttgart's terrain from flat to 1.5× to test the 151 m pressure limit	Zones switch from 1 to 2 exactly at 151 m
5	Wrote an exact ILP to check the greedy choice of backup pipes	Greedy is within 1.5 points of the optimum

The one-sentence message: the MST is a good starting point, but the cheapest network is a little longer than the MST and has shorter paths from the plant to the buildings.

2. Background: district heating as a graph problem

In a district heating network, a central plant heats water to about 110 °C. Pumps push it through a **supply pipe** to **substations**, each of which serves a block of buildings through a heat exchanger. The water comes back at about 50 °C through a parallel **return pipe**. Supply and return are laid together in one trench, so in our model one "edge" means one trench with two pipes.

Choosing the layout fixes almost everything about the network's life:

- **Capital cost**: digging trenches and buying pipe. Longer and fatter pipes cost more.

- **Pumping cost**: electricity to push water against friction. Longer paths and narrower pipes need more pump pressure.
- **Heat loss**: hot pipes lose heat to the ground along their whole length.
- **Feasibility**: on hills, water pressure at the lowest point can exceed what the pipe is rated for.
- **Reliability**: how much heat is lost when a pipe breaks.

Why a **tree**? A tree is the cheapest way to connect n nodes: exactly n − 1 edges and one path between any two nodes. With 41 nodes (1 plant + 40 substations) there are 820 possible edges, and by Cayley's formula 41^{39} possible spanning trees, far too many to try one by one. So we need algorithms.

Where this sits in the course syllabus:

Course topic	Where it appears in this project
Graph theory fundamentals, matrix representations	The network is a weighted graph; adjacency and Laplacian matrices
Paths, cycles, connected components	Trees have no cycles; a pipe failure splits the tree into two components
Centrality (degree, betweenness)	Which pipe carries the most flow and hurts most if it fails
Community structure, spectral bisection	Splitting the network into pressure zones
Random network models (null models)	Prelim: comparing our trees with random trees
Python libraries (NetworkX)	Cross-checking our own Kruskal and Prim implementations

3. The data and how the network is built

AixDHN (from RWTH Aachen) is a public dataset of 1,380 potential district-heating areas in Germany. For each area it gives the area outline, the total annual heat demand, the number of households, and the list of **100 m × 100 m census cells** that make it up. It does **not** contain pipes, diameters or demand per building. We found that out at the start, so we had to build the network ourselves.

NASA SRTM is a satellite elevation map of the Earth at about 30 m resolution. We used it to know the ground height along every possible pipe.

	Bremen	Stuttgart
Terrain	Flat (North German Plain)	Hilly (Neckar valley)
Annual heat demand	611,716 MWh/yr	247,766 MWh/yr
Households	38,794	28,520
Census cells	1,508	840
Height range	63 m	282 m

How the nodes are made. We group the census cells into substations with **k-means clustering**. With 40 substations, k-means splits the cells into 40 groups of nearby cells and places a substation at the centre of each group. The total demand of the district is shared between substations in proportion to how many cells each one serves (all cells are the same size, and the data has nothing finer). The

plant is placed at the census cell closest to the centre of all demand.

Annual energy is turned into a design (peak) load with **1,800 full-load hours**, the usual figure for German homes:

```
design load [kW] = annual energy [kWh] / 1,800 h
```

Why "120 networks" is not new data. k-means starts from random positions, so a different *random seed* gives a slightly different set of substations. After the prelim we built the network with 6 sizes (20, 40, 60, 80, 120, 160 substations) and 10 seeds each, in both cities: 6 × 10 × 2 = 120 networks from the same two districts. They test whether a result depends on where we happened to put the substations.

4. Edge weights: map distance and terrain

Every pair of nodes is a candidate pipe, so the graph is **complete**. What matters is the weight we put on each edge, because the MST minimises the total weight.

Map distance (the paper's choice). The weight is the straight-line distance on a map. On flat ground this is fine.

Terrain weight (our extension). On a hill, three things change: the pipe is longer than the map distance, digging on a slope costs more per metre, and very steep ground cannot be trenched at all. We sample the ground height every 25 m along the straight route and add up:

```
w(edge) = Σ over 25 m pieces of   √(Δx² + Δz²) × f(s)

f(s) = 1 + 1.176·s + 3.968·s²      s = slope = rise/run
```

f(s) is a cost multiplier fitted to industry figures for pipeline construction: flat ground 1.0, rolling ground about 1.6 at 15°, mountains about 3.0 at 30°. Slopes above 0.25 also get a ×1.35 rock-excavation factor, and routes steeper than about 31° are treated as impossible.

Real-world picture: think of two roads between two villages, one straight over a hill and one slightly longer around it. On a map the straight one is shorter, but digging a trench up and down the hill costs more per metre, so the road around can be cheaper. That is exactly what terrain weighting captures.

5. The tree algorithms

5.1 Kruskal's algorithm (what we use for the MST)

Sort all edges from lightest to heaviest. Go through them in order and accept an edge only if its two ends are not already connected. Stop when you have n − 1 edges. The check "already connected?" is done with a **union-find** (disjoint set) structure.

C++ view

Union-find is the classic DSU you may know from competitive programming: a parent array with path compression and union by rank. Our Python version is exactly this. Each find is almost O(1), so Kruskal is dominated by the sort: O(E log E).

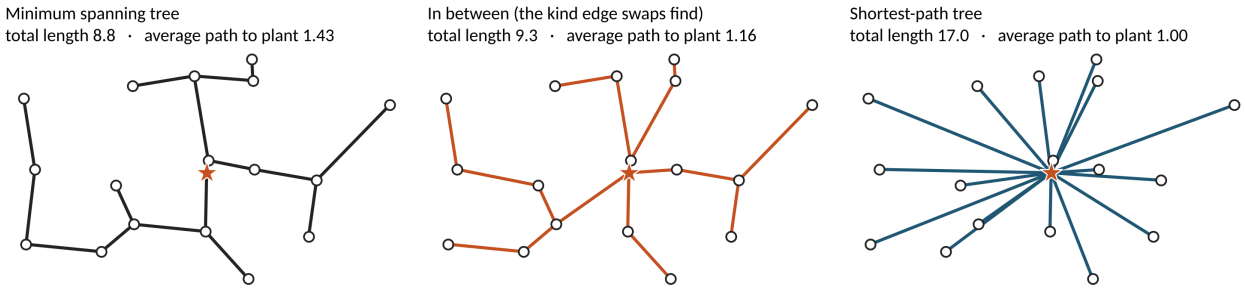
```
struct DSU {
    std::vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { std::iota(p.begin(), p.end(), 0); }
    int find(int a) { return p[a] == a ? a : p[a] = find(p[a]); } // path compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;     // already connected: edge would make a cycle
        if (r[a] < r[b]) std::swap(a, b);
        p[b] = a; if (r[a] == r[b]) r[a]++;   // union by rank
        return true;
    }
};
// Kruskal: sort(edges by weight), then
// for (auto [w,u,v] : edges) if (dsu.unite(u,v)) tree.push_back({u,v});
```

Prim's algorithm grows one tree from the plant, always adding the cheapest edge leaving it. We implemented a heap version and a dense $O(V^2)$ array version. At the prelim we showed a benchmark: on complete graphs the **dense array Prim** is fastest (28.8 ms at 1,600 nodes), Kruskal is second (420 ms, because it stops after checking only 0.7% of the sorted edges), and heap Prim is slowest (6.1 s). All three give the same tree, and we cross-checked them against NetworkX.

5.2 Other trees we compare

- **Star**: every substation gets its own pipe straight from the plant. Shortest possible path for every building, but a huge total length. It is the most expensive layout.
- **Steiner trees (ESMT and RSMT)**: allowed to add extra junction points ("Steiner points") that are not substations. This can shorten the tree. ESMT uses straight lines; RSMT uses only north-south and east-west lines (like streets on a grid). We use a heuristic called iterated 1-Steiner.
- **Shortest-path tree**: every node joins the plant by its shortest route. On a complete graph with straight-line weights this is the star.

5.3 MST versus shortest-path tree: the key picture for slides 4 and 5



Toy example with 17 substations. Left: the MST has the shortest total length but long, winding paths to the plant (star). Right: the shortest-path tree gives every node a direct route but uses almost twice the pipe. The middle tree is in between: a little longer than the MST, with clearly shorter paths. Our real result is that the cheapest network looks like the middle one.

The MST minimises **the sum of all edge lengths**. It does not care how far a building is from the plant *along the tree*. The shortest-path tree minimises **the path length to each node** and does not care about total length. Real cost depends on both, which is why neither extreme is the cheapest.

6. The cost model (the physics)

Every tree is judged by its **total annual cost**, divided by the heat delivered, giving **EUR per MWh**. Dividing by heat delivered lets us compare networks of different sizes. The model has four parts.

6.1 Flow in each pipe

In a tree there is only one path from the plant to each substation, so the heat flowing through a pipe is simply the total demand of everything downstream of it (its **subtree**). Pipes near the plant carry a lot, leaf pipes carry little. Heat flow is turned into water mass flow:

```
ṁ = P / (cp × ΔT)        cp = 4,229 J/(kg K),   ΔT = 110 − 50 = 60 °C
```

6.2 Pipe size

Real networks buy standard pipe sizes (the EN 253 catalogue, DN20 to DN1000), not any diameter. For each pipe we choose the **smallest** size for which the water speed is at most 3 m/s (noise and erosion) and the pressure drop is at most 100 Pa per metre.

6.3 Capital cost

```
capital per metre = 212 + 4,790 × (outer pipe diameter in m)   EUR/m
annual capital    = Σ capital per metre × terrain-weighted length × CRF
```

The two constants come from Persson and Werner (2011), a standard European cost function for district heating trenches.

The **capital recovery factor (CRF)** turns a one-time cost into an equal yearly payment, like a loan instalment: 4% interest over 30 years gives CRF ≈ 0.058.

6.4 Pumping cost

Friction in each pipe is found with the **Darcy–Weisbach** equation, with the friction factor from **Colebrook–White** (it depends on the Reynolds number and pipe roughness):

```
Δp = f × (L / D) × ρu²/2
```

The pump must overcome the worst path from the plant to any substation, supply plus return. **Height does not add pumping cost**: in a closed loop the water that goes up a hill comes back down in the return pipe, so the height cancels. (This was one of the bugs we caught early.) Pump power and yearly cost:

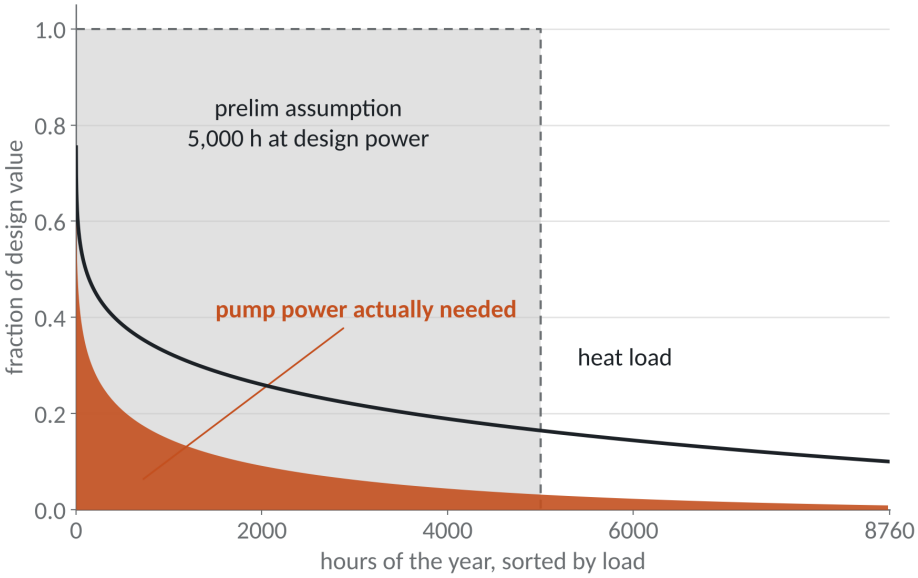
```
P_pump = ṁ_total × Δp_pump / (ρ × 0.75)
pumping cost = P_pump × operating hours × 0.1922 EUR/kWh
```

6.5 Heat loss

Each buried twin pipe loses heat to the soil through its foam insulation. The loss per metre comes from a standard thermal-resistance model (insulation, soil depth 0.8 m, ground at 8 °C, and the supply and return pipes warming each other). The lost heat is valued at 0.055 EUR/kWh, the cost of replacing it.

6.6 Design load versus seasonal load

At the prelim the pumps were costed at **full design power for 5,000 hours a year**. That is too much. Heating demand is high in winter and low in summer. After the prelim we used a **load-duration curve** (the Sochinsky formula): all 8,760 hours of the year sorted from highest load to lowest. Because pump power goes roughly with the **cube** of the flow, halving the flow cuts pump power to about one eighth. Integrated over the year, pump energy is about 8 times lower than the prelim assumed: the same as about 580 hours at full power instead of 5,000. The supply temperature is also lowered from 110 °C to 80 °C in mild weather, which reduces heat loss by about 18%.



Grey box: the prelim assumption. Black line: the heat load through the year. Orange: the pump power actually needed.

All costs on slides 4 to 7 use this seasonal model. The summary of what changes:

Stuttgart, terrain MST, 40 substations	Prelim (design load)	Seasonal
Pump energy	5,399 MWh/yr	631 MWh/yr
Heat loss	15,141 MWh/yr	12,357 MWh/yr
Total cost	22.39 EUR/MWh	18.07 EUR/MWh
Terrain saving vs map MST	3.0%	0.9%

7. Pressure zones

Water in a pipe going down a hill gains pressure: every 10 m of drop adds about 1 bar. Two limits apply to every point of the network:

- At the **highest** point the pressure must stay above the boiling pressure of 110 °C water (1.43 bar) plus a 0.5 bar safety margin, or the water flashes into steam.
- At the **lowest** point the pressure must stay below the pipe's rating. PN16 pipes are rated for 16 bar.

So the largest height difference one connected set of pipes (one **pressure zone**) can span is:

$$\Delta z_{max} = (16\ \text{bar} - 1.43\ \text{bar} - 0.5\ \text{bar}) / (\rho\, g) = 14.07 \times 10^5\ \text{Pa} / (951 \times 9.81) \approx 151\ \text{m}$$

With PN10 pipes the limit is 87 m, with PN25 it is 247 m. Stuttgart's pipe routes span 185 to 213 m, more than 151 m, so the network must be split into **2 zones** connected through a heat-exchanger station. Bremen's routes span only 15 to 30 m, so one zone is enough.

Where to split: at the prelim we placed zone boundaries with **spectral bisection**, straight from the course: the Fiedler vector (eigenvector of the second-smallest eigenvalue of the graph Laplacian) splits the graph into two balanced halves while cutting as few edges as possible. Every cut edge needs a heat-exchanger station, so fewer cuts is cheaper.

Real-world picture: a tall building's water supply. If you fed the whole building from one pump at the bottom, the bottom floors would see huge pressure. Instead tall buildings are split into zones with their own pumps or tanks. A hilly heating network works the same way.

8. Reliability: bridges, N-1 and expected unserved demand

A **bridge** is an edge whose removal disconnects the graph. In a tree, **every edge is a bridge**. If one pipe breaks, every substation downstream of it loses heat until it is repaired. Steiner trees do not help: they are still trees.

We measure reliability in two ways:

- **N-1 served**: take the single worst pipe failure, and ask what share of the demand still gets heat. For the Stuttgart terrain MST at the prelim this was 0%: one pipe next to the plant carried everything.
- **Expected unserved demand (EUD)**: for every pipe, failure rate × repair time (12 h) × demand cut off, added up. Failure rates come from published district heating data (Valinčius et al., 2015), are higher for small pipes (DN ≤ 150) and are adjusted by a corrosion model from the water speed and diameter.

$$\text{EUD} = \sum_{\text{pipes}} \lambda_{\text{pipe}} \times \text{repair time} \times \text{demand stranded by that pipe}$$

Adding backup pipes. Adding one non-tree edge creates one loop. Every tree edge on that loop stops being a bridge: if it breaks, heat can go the other way round. At the prelim we picked backup pipes **greedily**: each step adds the pipe with the best reduction in EUD per euro.

N-1 sizing. A loop only helps if each side can carry the full load when the other side is broken. So after adding loops we re-size every pipe for the worst single failure. Our first version forgot this and showed loops making the network cheaper (the flow split between two paths and pipes got smaller). That is physically wrong; with correct sizing the first loop costs about +4% in Stuttgart.

9. What we showed at the prelim

Results for one 40-substation network per city, design-load costing:

Topology	Bremen	Stuttgart
Star	147.9 km, 31.09 EUR/MWh	104.7 km, 43.72 EUR/MWh
MST (map distance)	49.0 km, 18.88 EUR/MWh	32.86 km, 23.09 EUR/MWh

Topology	Bremen	Stuttgart
MST (terrain-weighted)	same tree as map MST	32.89 km, 22.39 EUR/MWh (−3.0%)
ESMT	47.3 km, 18.03 EUR/MWh	31.41 km, 21.77 EUR/MWh
RSMT	49.1 km, 18.84 EUR/MWh	32.85 km, 20.87 EUR/MWh

- Terrain weighting changed nothing in flat Bremen (a good control) and saved 3.0% in Stuttgart with a tree that is slightly *longer*.
- Stuttgart needs 2 pressure zones (282 m of relief would mean 26 bar of static pressure, above PN16).
- In Stuttgart the first greedy backup pipe cost +4.3% and removed 57.2% of EUD; N-1 served rose from 0% to 68%.
- Reproducing Gao et al. (2020): we rebuilt their test case (33 users) to match both published pipe lengths exactly, and found their Table 2 is not self-consistent (the star and MST pipe costs need two different steam demands, 357 vs 87.5 kg/s).

The weak point, which we said ourselves: every number came from a single network per city, costed at design load.

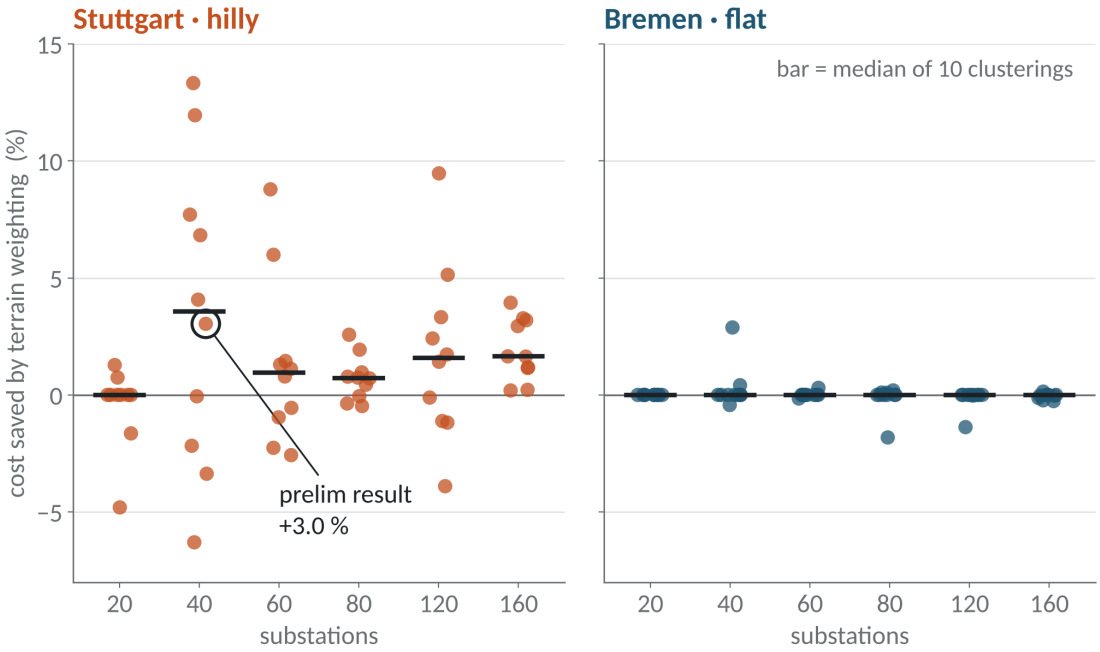
10. After the prelim: the five new results

This section follows the slides. For each one: the question, what we did, the result, how to say it, and what you may be asked.

Slide 3 · Result 1: the 3% terrain saving changes a lot from network to network

Question. Is the 3% saving from terrain weighting a reliable number, or did it depend on our one choice of substations?

What we did. Rebuilt both districts with 20, 40, 60, 80, 120 and 160 substations and 10 k-means seeds each (120 networks). For each one we built the map MST and the terrain MST and compared their costs with exactly the prelim model. We first checked that seed 42 with 40 substations gives the prelim numbers (23.09 and 22.39 EUR/MWh): it does.



Each dot is one network. The circled dot is the prelim result. Black bars are medians.

Result.

- Stuttgart, 40 substations: the saving ranges from **−6.3% to +13.3%** depending only on the seed.
- Over the 50 Stuttgart networks with 40 or more substations the median is **+1.2%**; it is negative in 15 of them.
- Bremen: zero in 33 of 50, as expected on flat ground.

Why so noisy? We split the prelim's 3.04% into cost parts: 2.68 points came from **pumping**, only 0.41 from cheaper **trenching** (heat loss went slightly the other way). Terrain weighting changed just 3 of 40 edges, and those 3 edges happened to shorten the main flow path, so the pump head fell from 16.3 to 14.2 bar. The edge weight never "aimed" at that: Kruskal only looks at each edge's own weight, while the flow in a pipe depends on the whole tree. With a different clustering the same kind of swap can lengthen the main path instead.

How to say it on stage

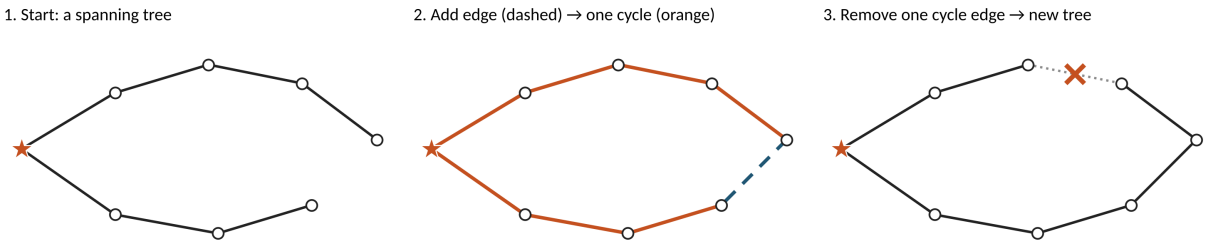
"At the prelim we said terrain saves 3 percent. We checked that on 120 networks. For the same city and size it goes from minus 6 to plus 13 percent, just because the substations moved. On average it is still positive in Stuttgart and zero in Bremen, so the idea holds, but the exact 3 percent was partly luck. The reason is that the MST only sees edge lengths, not the flow in the pipes."

Likely questions. "Where does the 120 come from?" (Section 3.) "Is the terrain effect real then?" Yes: the median is positive at every size from 40 up, Bremen stays at zero, and the relief sweep (slide 6) shows it growing with steepness.

Slide 4 · Result 2: edge swaps give a cheaper tree than the MST

Question. If the MST cannot see flow, can we find a better tree by optimising the real cost directly?

The move. A spanning tree has a useful property: if you add any one non-tree edge, it creates **exactly one cycle**. Remove any other edge of that cycle and you have a new spanning tree. This is called an **edge exchange** (or edge swap).



One edge swap. The tree stays a tree: same number of edges, everything still connected, no loops.

The search.

- Start from the MST.
- For each candidate edge (only between each node and its 8 nearest neighbours, to keep it fast), add it and try removing each edge of the cycle it closes.
- Evaluate every resulting tree with the **full cost model**: flows, pipe sizes, friction, pumping, heat loss and pressure zones are all recomputed.
- Keep the best swap if it lowers the cost. Repeat passes until no swap helps. This is a **local search**.

C++ view

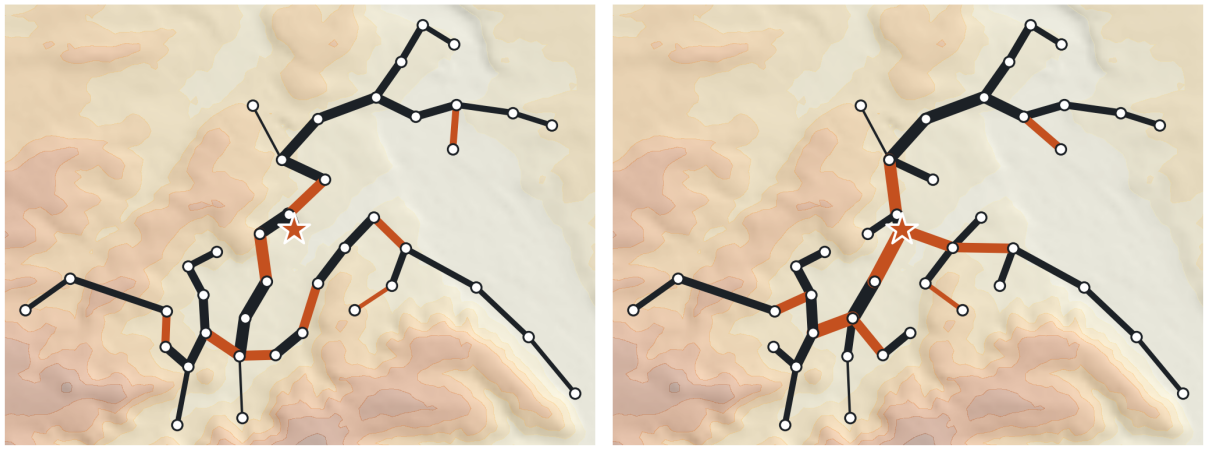
Think of it as hill-climbing over trees, similar to 2-opt for the travelling salesman: a neighbour of a tree is any tree one swap away, and we move to a cheaper neighbour until none exists. The cost function is a black box (the physics model), which is why we cannot use a greedy algorithm like Kruskal: the weight of an edge is not fixed, it depends on the rest of the tree.

```
tree = kruskal(G);  double best = cost(tree);
bool improved = true;
while (improved) {
    improved = false;
    for (auto [u,v] : candidate_edges) {     // 8 nearest neighbours per node
        if (tree.has(u,v)) continue;
        for (auto e : tree.path(u,v)) {       // the cycle closed by (u,v)
            auto t2 = tree.swap(e, {u,v});
            double c = cost(t2);              // full physics model
            if (c < best) { best = c; tree = t2; improved = true; }
        }
    }
}
```

Result (Stuttgart, seed 42, seasonal costing).

	Terrain MST	After swaps
Cost (EUR/MWh)	18.07	16.48
Pipe length (km)	32.9	33.9
Pump head (bar)	14.2	8.6
Longest path (hops)	21	15

Terrain MST **After flow-aware edge swaps**



Left: terrain MST. Right: after edge swaps. Orange edges are the ones that differ. Line thickness is pipe diameter, the star is the plant.

- **8.8% cheaper with 3.1% more pipe.** The new tree sends more branches straight out of the plant, so it is shallower.
- On 10 clusterings per city it improved **all 20 networks**: 5.8% on average in Stuttgart (range 0.8 to 9.3%), 6.7% in Bremen (5.1 to 8.4%).
- Bigger networks: 8.6% (Stuttgart) and 9.5% (Bremen) at 80 substations, 7.8% and 7.9% at 160 (the 160 case uses only 2 clusterings per city).

- Starting from the map MST or the terrain MST ends within 0.09 EUR/MWh, so the result does not depend much on the starting tree.
- It removes most of the clustering noise: in Stuttgart the spread across clusterings drops from 0.53 to 0.13 EUR/MWh (standard deviation).

The correction to our prelim. We had said "on flat ground the MST is the right answer". Edge swaps cut cost by 6.7% in flat Bremen, so that was wrong. The MST is the **shortest** tree, not the **cheapest**.

How to say it on stage

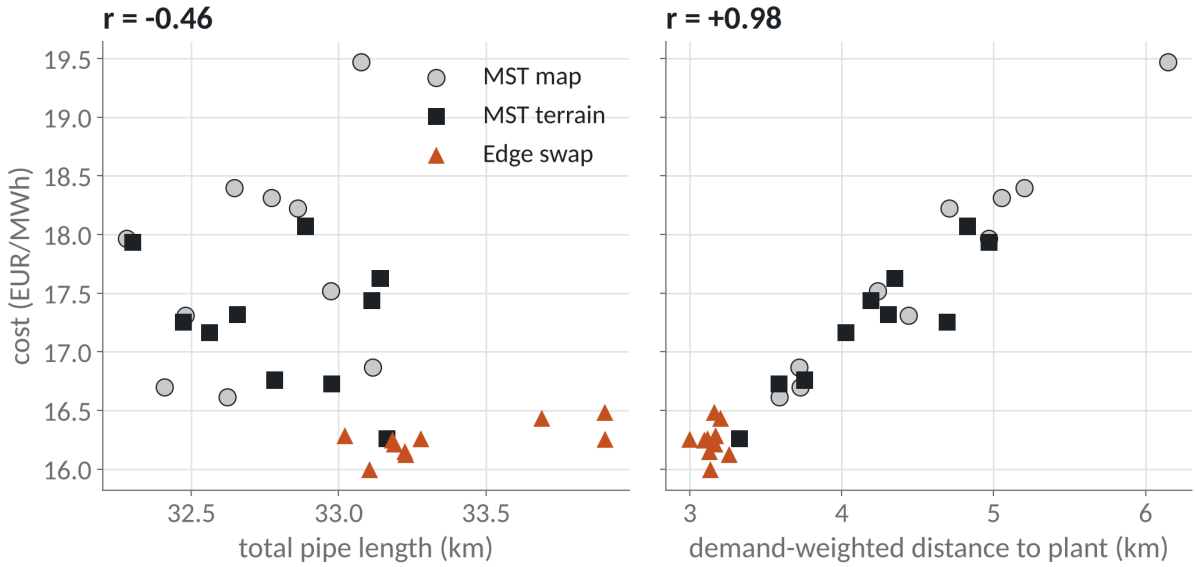
"Since the MST cannot see flow, we optimised the actual cost. We add one nearby edge, which makes a cycle, remove another edge from that cycle so it is a tree again, and keep the change if the cost goes down. The result is 3 percent longer but 9 percent cheaper. It was cheaper in all 20 networks we tried, and it works in flat Bremen too."

Slide 5 · Result 3: cost depends on distance to the plant, not total length

Question. Why is a longer tree cheaper? Which property of a tree actually decides its cost?

What we did. Took 30 trees for Stuttgart (10 clusterings × 3 trees: map MST, terrain MST, edge-swap tree) and compared their cost with two numbers:

- **Total pipe length**: what the MST minimises.
- **Demand-weighted distance to the plant**: for each substation, the distance along the tree from the plant to it, averaged with each substation weighted by its heat demand. It is the "average path length" of the tree as seen from the plant, counting big customers more.



Left: cost against total length. Right: cost against demand-weighted distance to the plant. r = correlation coefficient.

- Correlation with total length: **−0.46** (Stuttgart), −0.30 (Bremen). Not useful, even slightly the wrong way.
- Correlation with demand-weighted distance: **+0.98** (Stuttgart), +0.97 (Bremen).
- A straight-line fit of cost on length alone explains 21% of the variation ($R^2 = 0.21$). Adding the weighted distance explains 97%.

- Edge swaps cut the average depth from 5.7 to 3.9 hops.

Why physically. Heat that travels further passes through more pipe. The pipes near the plant carry the most flow, so they are the biggest and most expensive, and the pump has to push water along the longest path. A shallow tree keeps the heavy flow on short paths.

The graph-theory way to say it. The MST minimises total edge weight. A shortest-path tree minimises path length from the root. The cheapest network sits between the two (the picture in Section 5.3). This is a known trade-off in network design (for example the Prim–Dijkstra trade-off used in chip wiring), and our result shows it applies to district heating.

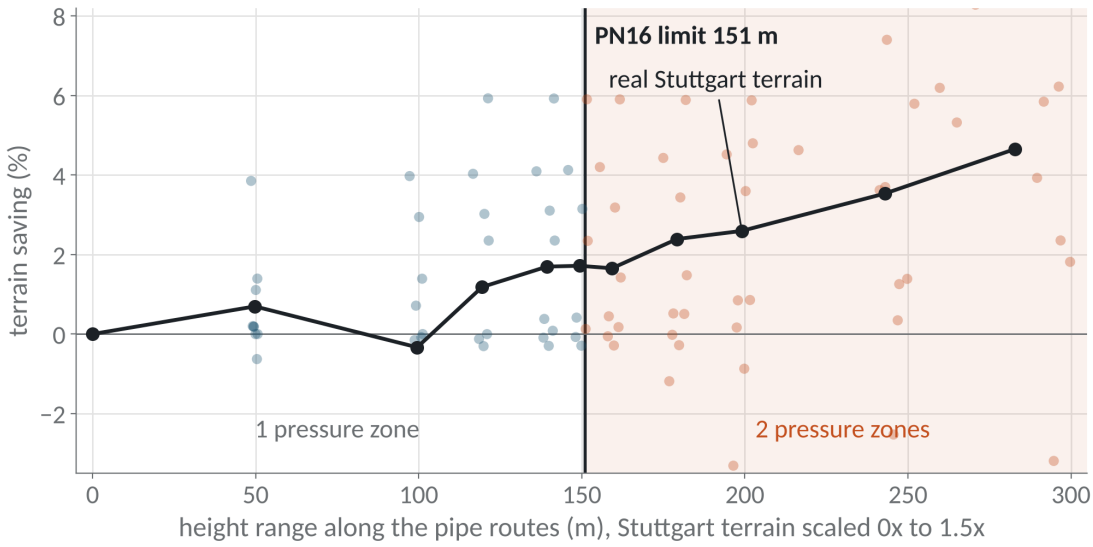
How to say it on stage

"Why is a longer tree cheaper? We took 30 trees and plotted cost. Against total length there is no clear relation. Against the average distance from the plant, weighted by demand, it is almost a straight line, correlation 0.98. So what matters is how far the heat travels, not how much pipe we lay."

Slide 6 · Result 4: pressure zones switch at 151 m, as predicted

Question. At the prelim we derived the 151 m one-zone limit and promised to test it on a third district. Does the number of zones really switch there?

What we did. We could not get elevation data for a third district in time, so we ran a **controlled test**: we took Stuttgart's elevation map and scaled every height about its mean by 0 (completely flat), 0.25, 0.5, 0.6, 0.7, 0.75, 0.8, 0.9, 1 (real), 1.25 and 1.5. Everything else stayed the same. For each scale we built 10 networks (40 substations, 10 clusterings) and recorded the zones and the terrain saving.



Each dot is one network. Blue: 1 zone. Orange: 2 zones. Black line: mean terrain saving at each scale.

- Below 151 m of relief along the pipe routes every network needs 1 zone; above 156 m all 10 need 2. Around 146 to 152 m, 3 of 10 need 2: exactly the borderline.
- With the real terrain, Stuttgart needs 2 zones in all 60 networks we built (all sizes and seeds) and Bremen needs 1 in all 60.

- The terrain saving grows with steepness: 0% when flat, 2.6% for the real terrain (mean of 10 clusterings, seasonal costing), 4.7% at 1.5×. So terrain weighting is a real effect even if one network is noisy.

Pressure class. With real terrain, PN10 pipes would need 3 zones, PN16 needs 2 and PN25 needs 1. In our cost model a zone station is cheap, so the cost changes by only about 0.5%. Pressure is a **feasibility** constraint, not a big cost item.

How to say it on stage

"We had planned to test the 151 metre limit on a third city but could not get the data. So we scaled Stuttgart's heights from flat to one and a half times. The number of zones switches exactly at 151 metres. The same sweep shows the terrain saving is zero when flat and grows as the ground gets steeper."

Slide 7 · Result 5: greedy backup pipes are close to the exact ILP optimum

Question. At the prelim we chose backup pipes greedily and promised to compare with an exact method. How far from optimal is greedy?

The key observation. On a tree, adding pipe (u, v) protects exactly the tree edges on the path from u to v (they now lie on a cycle). So each candidate pipe "covers" a set of tree edges, and each tree edge has a weight: the EUD it causes. Choosing k pipes to remove the most EUD is a **weighted maximum coverage** problem.

The ILP. One binary variable x per candidate pipe (1 = build it) and one binary variable y per tree edge (1 = protected):

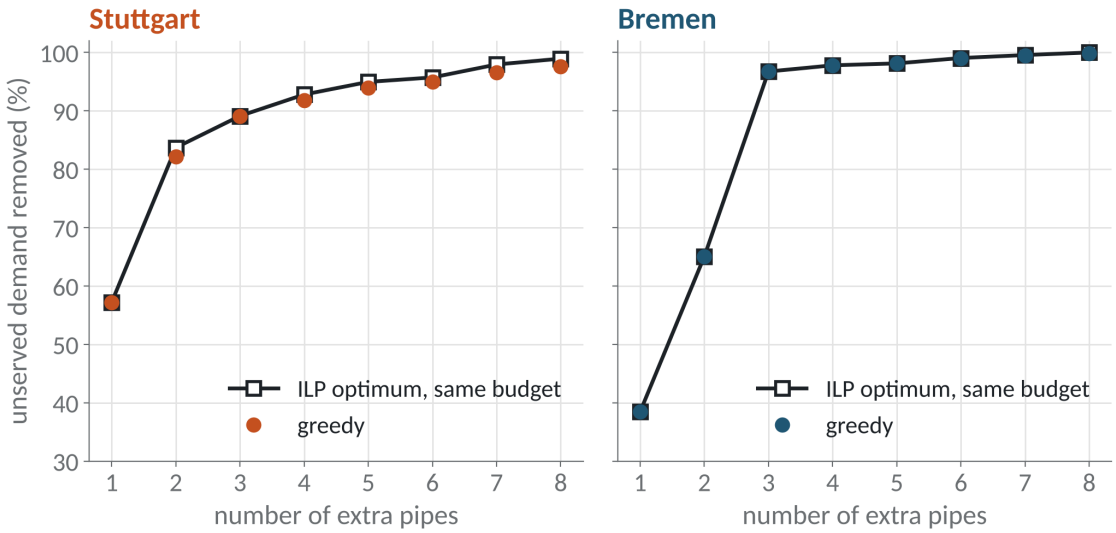
```
maximise   Σ_e w_e · y_e                    EUD removed
subject to y_e ≤ Σ (x_c for pipes c covering e)  e protected only if covered
           Σ_c x_c ≤ k                       at most k pipes
           Σ_c cost_c · x_c ≤ greedy budget  same money as greedy
           x, y ∈ {0, 1}
```

That is 780 candidate pipes + 40 tree edges = 820 binary variables. Solved with the HiGHS solver (through scipy) in under a second for k = 1 to 8.

C++ view

For k ≤ 3 we also checked by brute force. Each candidate pipe is stored as a 64-bit mask of the tree edges it covers, so combining pipes is just a bitwise OR, and the EUD removed is the weighted popcount. 780 singles, 303,810 pairs and 78.8 million triples per city run in seconds. The ILP matches the brute force exactly for k = 1, 2, 3.

```
uint64_t mask[780];    // bit e set if the pipe covers tree edge e (40 edges)
double gain(uint64_t m) {                  // EUD removed by a set of pipes
    double s = 0;
    for (int e = 0; e < 40; ++e) if (m >> e & 1) s += w[e];
    return s;
}
// for i < j < k:  best = max(best, gain(mask[i] | mask[j] | mask[k]));
```



Black line: ILP optimum for the same budget. Dots: greedy. 1 to 8 extra pipes.

- Greedy is never more than **1.5 percentage points** below the optimum, for 1 to 8 pipes, in both cities. So greedy is good enough.
- At the prelim, brute force showed greedy exactly on the optimum in 5 of 6 cases (k = 1, 2, 3 in two cities). The one miss (Stuttgart, 2 pipes) was 82.2% vs 83.7%, and that "better" pair costs much more once the network is properly sized for N-1 (+30.5% vs +13.2%).

Putting both ideas together (Stuttgart, mean of 10 clusterings, seasonal costing):

Design	EUR/MWh	N-1 served
Terrain MST	17.25	30%
Terrain MST + 1 loop	19.39	68%
Edge-swap tree	16.24	51%
Edge-swap tree + 1 loop	18.45	67%

The edge-swap tree plus one loop has the same reliability as the MST plus one loop and costs about 5% less (Bremen: 15.11 vs 15.95 EUR/MWh). Notice also that the edge-swap tree alone is more robust than the MST (51% vs 30%): because it is shallower, less demand hangs off any one pipe.

How to say it on stage

"A tree has no backup. We added backup pipes greedily at the prelim and promised to check against an exact optimisation. We wrote it as an integer linear program: pick the pipes that protect the most demand for the same budget. Greedy is never more than 1.5 points below the optimum. And the edge-swap tree plus one loop is as reliable as the MST plus one loop and about 5 percent cheaper."

11. Summary slide, limitations and the main message

At the prelim we said	Now	Status
Terrain MST saves 3%	True on average, but noisy for one network	Weaker
MST is best on flat ground	Edge swaps are 6 to 9% cheaper	Wrong

At the prelim we said	Now	Status
Stuttgart needs 2 pressure zones	Yes, and the 151 m limit is confirmed	Holds
Greedy extra pipe is good enough	Yes, within 1.5 points of the ILP	Holds

Main message: the MST is a good start, but the cheapest tree is a little longer and has shorter paths to the plant.

Limitations (say them before you are asked):

- The seasonal load curve is a standard formula, not measured hourly data. Changing the summer load from 5 to 20% did not change the conclusions.
- Edge swapping is a local search, so there is no proof it finds the best tree. Two different starting trees end almost at the same cost, which is a good sign.
- Only two districts. The relief sweep scales one city's terrain; a real third district would be a stronger test.
- The 160-substation edge-swap result uses only 2 clusterings per city.
- Pipes run in straight lines between substations; real pipes follow streets.
- Total pump head is not checked against the pipe pressure rating (the MSTs need 14 to 20 bar of pump head). The shallower edge-swap trees need much less, which helps.

Backup slides. Slide 9 (seasonal load model) and slide 10 (sensitivity and network size) are only for questions. Sensitivity: changing electricity price ±50%, heat price ±50%, discount rate 2 to 6% or the temperature to 90/45 °C moves the terrain saving between 1.9 and 4.0% but **never changes which layout is cheapest**.

12. Question bank for the Q&A

Q. Where did the data for the 120 networks come from?
The same data as the prelim: AixDHN (heat demand per census cell) and NASA SRTM elevation. We cluster the cells into substations with k-means. We repeated that with 6 sizes (20 to 160) and 10 random seeds each, in both cities. So these are 120 ways of dividing the same two districts, not 120 new districts. They test whether results depend on where the substations are placed.

Q. Why k-means?
It groups nearby census cells and puts each substation at the centre of its group, which is how a real substation serves a block of buildings. AixDHN has no substations, so we had to create them.

Q. Why does the clustering change the terrain saving so much?
Terrain weighting changes only a few edges. Whether those edges shorten or lengthen the main flow path depends on where the substations are. Most of the saving was pumping, which depends on the flow paths.

Q. What exactly is an edge swap? Is the result still a tree?
Add one non-tree edge, which creates exactly one cycle, and remove a different edge from that cycle. The number of edges stays $n - 1$ and everything stays connected, so yes, it is always a spanning tree.

Q. Is the edge-swap tree optimal?
Not guaranteed: it is a local search. But starting from two different MSTs gives almost the same cost (within 0.09 EUR/MWh), so it does not seem stuck in a bad local minimum.

Q. Why not use a shortest-path tree or a star?
They minimise path length but use far more pipe. The star was the most expensive layout (43.72 EUR/MWh in Stuttgart vs 22.39 for the terrain MST at the prelim). The cheapest tree is between the MST and the shortest-path tree.

Q. Why is the longer tree cheaper?
Cost follows the demand-weighted distance from the plant ($r = 0.98$), not total length. Shorter paths for the big flows mean smaller pump head and less pipe carrying large flows over long distances.

Q. Why not just use Kruskal with a better weight?
Because the right "weight" of a pipe depends on how much flow it carries, and that depends on the whole tree. No fixed edge weight can capture that, so a greedy algorithm like Kruskal cannot optimise it directly.

Q. How long does the edge-swap search take?
For 40 substations, about 1,300 to 1,800 cost evaluations and a few seconds. For 160 substations, one to two minutes on a laptop.

Q. Where does 151 m come from?
(16 bar − 1.43 bar boiling pressure at 110 °C − 0.5 bar margin) divided by ρg (951 kg/m^3 × 9.81) gives about 151 m of water column.

Q. Why scale the terrain instead of using a third city?
We could not get elevation data for a third district in time. Scaling one city's terrain is a controlled test: only the height changes and everything else stays fixed. A real third district would still be better; it is in our limitations.

Q. Why does height not increase pumping cost?
The network is a closed loop. Water pumped up a hill in the supply pipe comes back down in the return pipe, so the height difference cancels. Height matters for pressure limits, not pump energy.

Q. What is the ILP? Why 820 variables?
One binary variable per candidate backup pipe (780) and one per tree edge (40). An edge counts as protected if at least one chosen pipe closes a cycle through it. We maximise the EUD removed with the same number of pipes and budget as greedy. Solved with HiGHS in under a second.

Q. Why is greedy so good here?
Maximum coverage is a submodular problem: adding a pipe helps less once its edges are already covered. For the plain version (pick any k pipes), greedy is proven to reach at least 63% (1 − 1/e) of the optimum even in the worst case, and in practice it is usually much closer, as here (within 1.5 points).

Q. What is EUD?
Expected unserved demand: for each pipe, failure rate × repair time × demand cut off, summed. It weights each failure by how likely it is and how much heat it cuts off.

Q. What is N-1 sizing and why does it matter?
With a loop, each side must be able to carry the full load when the other side breaks. If you size pipes on the normal split flow, they come out too small and the loop looks free. That was a bug we found and fixed before the prelim.

Q. How realistic is the seasonal model?
It uses the standard Sochinsky load-duration curve fitted to 1,800 full-load hours, not measured data. We varied the summer load from 5 to 20% and the conclusions did not change.

Q. Why do Steiner trees not help reliability?
A Steiner tree is still a tree: every edge is a bridge. Steiner points actually concentrate flow into shared trunks, which makes single failures worse.

Q. What did you find about the reference paper?
At the prelim: their Table 2 is not self-consistent. The star and MST pipe costs require two different total steam demands (357 vs 87.5 kg/s) under their own equations.

Q. What would you do next?
Use a real third district; use street-following routes instead of straight lines; get measured hourly demand; and combine the edge swap and the loop choice into one optimisation instead of doing them one after the other.

13. Glossary

Term	Meaning
Spanning tree	A set of n − 1 edges that connects all n nodes with no cycles.
MST	Minimum spanning tree: the spanning tree with the smallest total edge weight.
Shortest-path tree	A tree where each node is joined to the root by its shortest possible path.
Steiner tree (ESMT, RSMT)	A tree allowed extra junction points. E = Euclidean (straight lines), R = rectilinear (grid lines).
Bridge	An edge whose removal disconnects the graph. Every edge of a tree is a bridge.
Edge swap / edge exchange	Add a non-tree edge (makes one cycle) and remove another edge of that cycle.
k-means	Clustering that splits points into k groups around k centres.
Substation	Where the network hands heat to a group of buildings through a heat exchanger.
DN	Nominal pipe diameter in mm, from the standard catalogue (DN20 to DN1000).
PN16	Pipe pressure class: rated for 16 bar.
Pressure zone	Part of the network that is hydraulically separate, joined to the rest through a heat exchanger.
Darcy–Weisbach, Colebrook–White	The standard equations for friction pressure loss in a pipe and its friction factor.

Term	Meaning
CRF	Capital recovery factor: turns a one-off investment into an equal yearly cost.
Full-load hours	Annual energy divided by peak load. 1,800 h for German homes.
Load-duration curve	All hours of the year sorted from highest to lowest heat load.
EUD	Expected unserved demand: likely heat not delivered per year because of pipe failures.
N-1	The network must keep working when any one component fails.
ILP	Integer linear program: linear optimisation where variables must be whole numbers (here 0 or 1).
Fiedler vector	Eigenvector of the second-smallest eigenvalue of the graph Laplacian; used for spectral bisection.
Correlation r	From −1 to +1: how close two quantities are to a straight-line relation.

14. One-page cheat sheet

Slide	One line to remember	Numbers
2 Prelim and after	Prelim used one network per city; we did 5 things after	3%, 2 zones, 57%
3 120 networks	Terrain saving is right on average but noisy; MST cannot see flow	−6% to +13%; median +1.2%; 2.7 of 3.0 points was pumping
4 Edge swaps	Add an edge, remove one from the cycle, keep if cheaper	18.07 → 16.48 EUR/MWh; +3% length; −9% cost; 20/20 networks
5 What decides cost	Distance from the plant, not total length	r = −0.46 vs +0.98; R^2 0.21 → 0.97
6 Pressure zones	Scaled terrain; zones switch at 151 m	PN10 87 m, PN16 151 m, PN25 247 m; saving 0 → 2.6 → 4.7%
7 ILP vs greedy	Greedy is good enough; swap tree + 1 loop is cheaper	gap ≤ 1.5 points; 820 variables; 18.45 vs 19.39
8 Summary	MST is a good start; cheapest tree is a bit longer and shallower	Weaker, Wrong, Holds, Holds

Timing (about 5 min 45 s): title 10 s · slide 2 40 s · slide 3 50 s · slide 4 60 s · slide 5 50 s · slide 6 50 s · slide 7 50 s · summary 35 s.

Presenting: open the file, click Enable Editing, press Alt + F5 for Presenter View. Projector: Windows + P → Extend.

If you blank on a slide: say what the question was, what we did, and the one number in bold on the slide. That is enough for every slide.